1.Inventory and Warehouse Optimization using Generic Algorithm for optimization

```
import random
random.seed(42)
products = ["Laptop", "Phone", "Tablet", "Keyboard", "Monitor"]
demand = [40, 60, 35, 80, 30]
holding_cost = [8, 4, 5, 2, 6]
shortage_penalty = [50, 30, 35, 15, 40]
warehouses = ["Warehouse A", "Warehouse B", "Warehouse C"]
capacity = [100, 120, 90]
transport_cost = [
  [4, 6, 5, 3, 7],
  [5, 3, 6, 4, 5],
  [6, 5, 3, 5, 4]
]
POPULATION_SIZE = 100
GENERATIONS = 200
MUTATION_RATE = 0.10
CROSSOVER_RATE = 0.80

NUM_PRODUCTS = len(products)
NUM_WAREHOUSES = len(warehouses)
NUM_GENES = NUM_PRODUCTS * NUM_WAREHOUSES

def create_solution():
  chromosome = [0] * NUM_GENES
  remaining_capacity = capacity[:]
  product_order = list(range(NUM_PRODUCTS))
  random.shuffle(product_order)
  for p in product_order:
    remaining_demand = demand[p]
    warehouse_order = list(range(NUM_WAREHOUSES))
```

```
        random.shuffle(warehouse_order)

        for w in warehouse_order:
            quantity = random.randint(
                0, min(remaining_demand, remaining_capacity[w])
            )
            chromosome[w * NUM_PRODUCTS + p] = quantity
            remaining_capacity[w] -= quantity
            remaining_demand -= quantity
            if remaining_demand == 0:
                break
    return chromosome

def repair(chromosome):
    chromosome = [max(0, int(g)) for g in chromosome]
    for p in range(NUM_PRODUCTS):
        total = sum(
            chromosome[w * NUM_PRODUCTS + p]
            for w in range(NUM_WAREHOUSES)
        )
        while total > demand[p]:
            w = random.randrange(NUM_WAREHOUSES)
            index = w * NUM_PRODUCTS + p
            if chromosome[index] > 0:
                chromosome[index] -= 1
                total -= 1
    for w in range(NUM_WAREHOUSES):
        start = w * NUM_PRODUCTS
        total = sum(chromosome[start:start + NUM_PRODUCTS])
        while total > capacity[w]:
            p = random.randrange(NUM_PRODUCTS)
```

```
            index = w * NUM_PRODUCTS + p

            if chromosome[index] > 0:
                chromosome[index] -= 1
                total -= 1
    return chromosome
def calculate_cost(chromosome):
    holding = 0
    transportation = 0
    shortage = 0
    for w in range(NUM_WAREHOUSES):
        for p in range(NUM_PRODUCTS):
            quantity = chromosome[w * NUM_PRODUCTS + p]
            holding += quantity * holding_cost[p]
            transportation += quantity * transport_cost[w][p]
    for p in range(NUM_PRODUCTS):
        supplied = sum(
            chromosome[w * NUM_PRODUCTS + p]
            for w in range(NUM_WAREHOUSES)
        )
        shortage_quantity = max(0, demand[p] - supplied)
        shortage += shortage_quantity * shortage_penalty[p]
    return holding + transportation + shortage
def fitness(chromosome):
    return 1 / (1 + calculate_cost(chromosome))
def select_parent(population):
    candidates = random.sample(population, 3)
    return max(candidates, key=fitness)
def crossover(parent1, parent2):
    if random.random() > CROSSOVER_RATE:
        return parent1[:], parent2[:]
```

```
    point = random.randint(1, NUM_GENES - 1)
    child1 = parent1[:point] + parent2[point:]
    child2 = parent2[:point] + parent1[point:]
    return repair(child1), repair(child2)
def mutate(chromosome):
    chromosome = chromosome[:]
    for i in range(NUM_GENES):
        if random.random() < MUTATION_RATE:
            chromosome[i] = random.randint(0, max(demand))
    return repair(chromosome)
def genetic_algorithm():
    population = [
        create_solution() for _ in range(POPULATION_SIZE)
    ]
    best_solution = min(population, key=calculate_cost)
    for generation in range(GENERATIONS):
        population.sort(key=calculate_cost)
        new_population = [best_solution[:]]
        while len(new_population) < POPULATION_SIZE:
            parent1 = select_parent(population)
            parent2 = select_parent(population)
            child1, child2 = crossover(parent1, parent2)
            child1 = mutate(child1)
            child2 = mutate(child2)
            new_population.append(child1)
            if len(new_population) < POPULATION_SIZE:
                new_population.append(child2)
        population = new_population
        current_best = min(population, key=calculate_cost)
        if calculate_cost(current_best) < calculate_cost(best_solution):
```

```
            best_solution = current_best[:]

        if (generation + 1) % 50 == 0:
            print(
                "Generation:", generation + 1,
                "Best Cost:", calculate_cost(best_solution)
            )
    return best_solution
def display_results(solution):
    print("\nINVENTORY AND WAREHOUSE OPTIMIZATION")
    print("-" * 65)
    print(f"{'Product':<12}", end="")
    for warehouse in warehouses:
        print(f"{warehouse:<16}", end="")
    print(f"{'Total':<10}{'Demand':<10}")
    for p, product in enumerate(products):
        print(f"{product:<12}", end="")
        total = 0
        for w in range(NUM_WAREHOUSES):
            quantity = solution[w * NUM_PRODUCTS + p]
            total += quantity
            print(f"{quantity:<16}", end="")
        print(f"{total:<10}{demand[p]:<10}")
    print("\nWAREHOUSE CAPACITY")
    for w, warehouse in enumerate(warehouses):
        total = sum(
            solution[w * NUM_PRODUCTS + p]
            for p in range(NUM_PRODUCTS)
        )
        print(
            f"{warehouse}: {total}/{capacity[w]} units used"
```

```
        )

    print("\nCOST DETAILS")
    holding = sum(
        solution[w * NUM_PRODUCTS + p] * holding_cost[p]
        for w in range(NUM_WAREHOUSES)
        for p in range(NUM_PRODUCTS)
    )
    transportation = sum(
        solution[w * NUM_PRODUCTS + p] * transport_cost[w][p]
        for w in range(NUM_WAREHOUSES)
        for p in range(NUM_PRODUCTS)
    )
    shortage = sum(
        max(
            0,
            demand[p] - sum(
                solution[w * NUM_PRODUCTS + p]
                for w in range(NUM_WAREHOUSES)
            )
        ) * shortage_penalty[p]
        for p in range(NUM_PRODUCTS)
    )
    print("Holding Cost:", holding)
    print("Transportation Cost:", transportation)
    print("Shortage Penalty:", shortage)
    print("Total Cost:", calculate_cost(solution))

if __name__ == "__main__":
    best_solution = genetic_algorithm()
    display_results(best_solution)
```

```
PS C:\Users\harsh\OneDrive\Desktop\CN> python generic.py
Generation: 50 Best Cost: 1928
Generation: 100 Best Cost: 1923
Generation: 150 Best Cost: 1906
Generation: 200 Best Cost: 1904

INVENTORY AND WAREHOUSE OPTIMIZATION
----------------------------------------------------------------
Product     Warehouse A     Warehouse B     Warehouse C     Total     Demand
Laptop      39              1               0               40        40
Phone       0               60              0               60        60
Tablet      0               0               35              35        35
Keyboard    57              23              0               80        80
Monitor     0               0               30              30        30

WAREHOUSE CAPACITY
Warehouse A: 96/100 units used
Warehouse B: 84/120 units used
Warehouse C: 65/90 units used

COST DETAILS
Holding Cost: 1075
Transportation Cost: 829
Shortage Penalty: 0
Total Cost: 1904
PS C:\Users\harsh\OneDrive\Desktop\CN>
```

Genetic Algorithm for optimization.

Algorithm :-

Step:- 1. Define the objective function
2. Set GA parameters population size, Number of generations, crossover rate, mutation rate.
3. Generate an initial population of random binary chromosomes.
4. Decode each chromosomes into a decimal value.
5. Calculate the fitness using $f(x) = x^2$.
6. Select parents based on their fitness.
7. Perform crossover to generate offspring.
8. Apply mutation to some bits of the offspring.
9. Form the new population.
10. Repeat step 4-9 for the specified No. of generations.
11. Find & Display the chromosome having the highest fitness
12. Display the corresponding value of x and maximum function value.

Step:13. End

Inventory & Warehouse Optimization.

Problem statement :- inventory & warehouse management is an important process in supply chain management. maintaining excess inventory increases storage costs, while inefficient inventory can lead to stock shortages & delay customer order.

Date __/__/__
Page _____

objectives :-

* To minimize total inventory costs, including ordering, holding & shortage costs.
* To allocate warehouse storages & excess inventory.
* To use a Genetic Algorithm to find an effective inventory & warehouse management strategy.

BEGIN.
    initialize population P randomly
    For each solution S in P Do.
        calculate inventory cost
        calculate warehouse storage cost.
        calculate storage cost
        calculate total cost of S.
    END FOR.
Find the Best solution in P.
    while termination condition is not satisfied do.
    Select parent solutions from P.
        crossover = parents
            generate.offspring().
            Apply mutation to offspring.
            FOR each offspring S DO.
            check inventory constraints
            check warehouse capacity constraints.
        If constraints are satisfied then
            Calculate total cost of S
        else
            reject S.
        END IF
END FOR.

Create a new population using offspring and the best solutions from the old population.
Update the best solution.
END WHILE.
RETURN best inventory & warehouse allocation plan
END.

OUTPUT :-
Generation : 50 Best cost : 1928
Generation : 100 Best cost : 1923.
Generation : 150 Best cost : 1906.
Generation : 200 Best cost : 1904.

INVENTORY & WAREHOUSE OPTIMIZATION.

Product	warehouseA	B	C	Total	Demand.
Laptop	39	1	0	40	40
Phone	0	60	0	60	60
tablet	0	0	35	35	35
Keyboard	57	23	0	80	80
Monitor	0	0	30	30	30.

Warehouse Capacity.
Warehouse A : 96/100 units used
warehouse B : 84/120 unit used
warehouse C : 65/90 unit used.
Holding cost : 1075
Transportation cost : 829
Shortage Penalty : 0.
Total cost : 1904.

8
16/10/26.